Южно-Уральский государственный университет
Национальный исследовательский университет

НТЦ ПОЛИТЕХНИКА
технологии • решения • интеграция

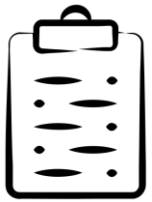
3.3. JavaScript. Основы DOM-модели

2026 ООО НТЦ «Политехника»
Челябинск, ул. Карла Маркса, д. 38

Кузнецов Сергей
Фоменко Алексей

DOM - Объектная модель документа

НТЦ ПОЛИТЕХНИКА

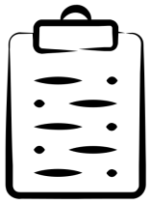


- Объектная Модель Документа (DOM) – это программный интерфейс (API) для HTML и XML документов.
- DOM предоставляет структурированное представление документа и определяет то, как эта структура может быть доступна из программ, которые могут изменять содержимое, стиль и структуру документа.
- В соответствии с DOM каждый HTML-тег является объектом. Вложенные теги являются дочерними к родительскому элементу.

```
API (веб либо XML страница)
=
DOM + JS (язык описания скриптов)
```

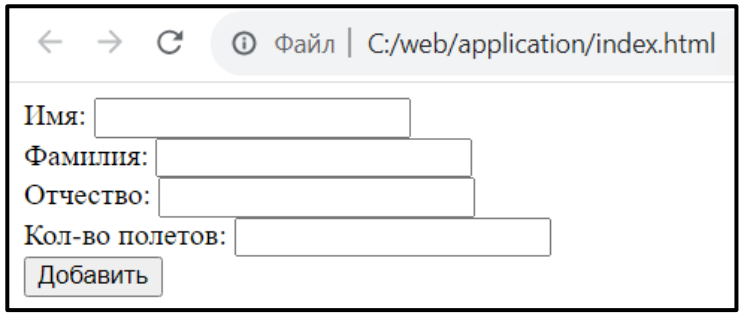
DOM – Корневой элемент

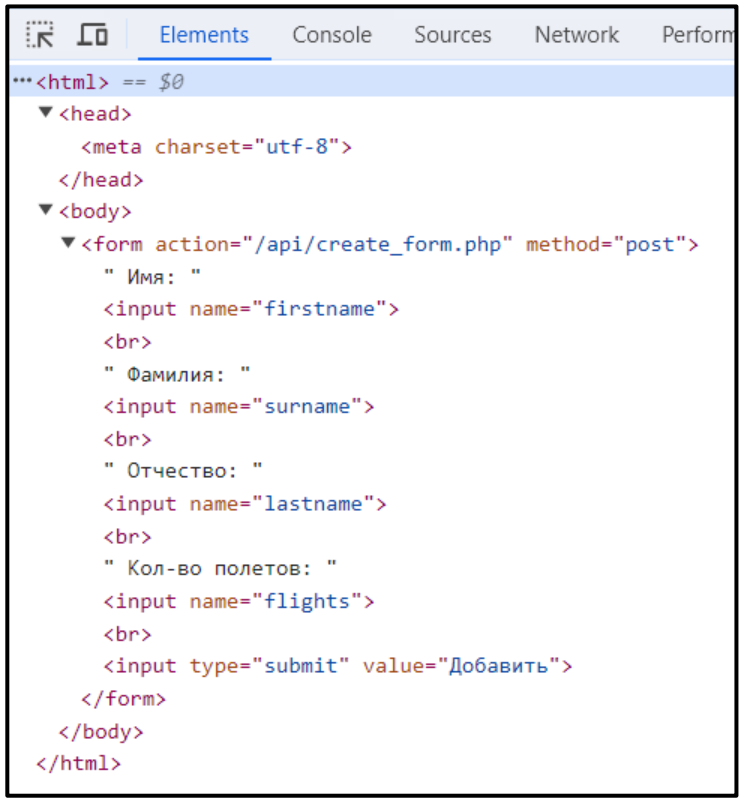
НТЦ ПОЛИТЕХНИКА



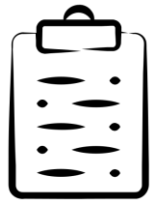
- DOM – это представление HTML-документа в виде дерева тегов
- Теги являются узлами-элементами (или просто элементами). Они образуют структуру дерева: <html> – это корневой узел, <head> и <body> его дочерние узлы и т.д.

```
<meta charset="utf-8"/>
<form action="/api/create_form.php" method="post">
    Имя: <input name="firstname"><br>
    Фамилия: <input name="surname"><br>
    Отчество: <input name="lastname"><br>
    Кол-во полетов: <input name="flights"><br>
    <input type="submit" value="Добавить">
</form>
```





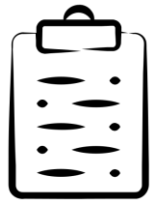
DOM – Корневой элемент



- Каждая веб-страница, которая загружается в браузер, имеет свой собственный объект document, доступный из JS
- document.body это элемент, который включает в себя содержимое страницы.

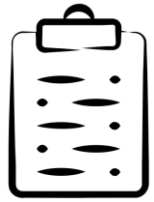
```
> let body = document.body
< undefined
> typeof (body)
< 'object'
```

JS. Получение элементов. querySelector



document метод querySelector() возвращает первый элемент (Element) документа, который соответствует указанному селектору или группе селекторов:

```
element = document.querySelector(selectors);
```



Селекторы такие же, как и в CSS, например:

```
document.querySelector('тег')
document.querySelector('#идентификатор')
document.querySelector('.класс')
document.querySelector('[атрибут]')
document.querySelector('[атрибут="значение"]')
```

JS. Получение элементов. querySelector

НТЦ ПОЛИТЕХНИКА

```
<meta charset="UTF-8">
<body>
    <div class="ex">Пример 1</div>
    <span>Пример 2</span>
    <div id="someItem">Пример 3</div>
    <span class="ex">Пример 4</span>
    <br/>
    <input type="text" value="Пример 5">
</body>
```

```
document.querySelector('div')
document.querySelector('span')
document.querySelector('#someItem')
document.querySelector('span.ex')
document.querySelector('br')
document.querySelector('[type="text"]')
```

```
> document.querySelector('div')
<- <div class="ex">Пример 1</div>
```

```
> document.querySelector('span')
<- <span>Пример 2</span>
```

```
> document.querySelector('#someItem')
<- <div id="someItem">Пример 3</div>
```

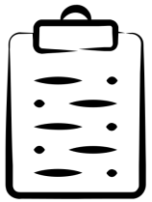
```
> document.querySelector('span.ex')
<- <span class="ex">Пример 4</span>
```

```
> document.querySelector('br')
<- <br>
```

```
> document.querySelector('[type="text"]')
<- <input type="text" value="Пример 5">
```

JS. querySelector. Вложенность

НТЦ ПОЛИТЕХНИКА



- Можно комбинировать селекторы.
- querySelector будет искать элементы во всех вложенных дочерних элементах, пока не найдет нужный.

```
<meta charset="UTF-8">
<body>
    <div class="ex">
        <input type="text" value="Пример 1">
        <span>Пример 2</span>
        <div>
            <input type="text" value="Пример 3">
        </div>
    </div>
</body>
```

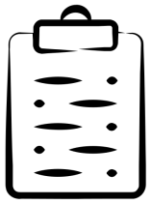
```
> document.querySelector('input')
<-   <input type="text" value="Пример 1">
```

```
> document.querySelector('div.ex input')
<-   <input type="text" value="Пример 1">
```

```
> document.querySelector('div.ex div input')
<-   <input type="text" value="Пример 3">
```

JS. querySelector. Вложенность

НТЦ ПОЛИТЕХНИКА



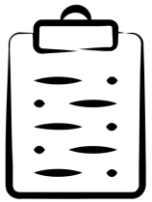
Метод querySelector можно вызывать не только у document, а у любого элемента

```
<meta charset="UTF-8">
<body>
    <div id="div1">
        <span>Пример 1</span>
    </div>
    <div id="div2">
        <span>Пример 2</span>
    </div>
</body>
```

```
> let elem = document.querySelector('#div1')
<- undefined
> elem.querySelector('span')
<-   <span>Пример 1</span>
```

```
> let span2 = document.querySelector('#div2').querySelector('span')
<- undefined
> span2
<-   <span>Пример 2</span>
```

JS. Получение по идентификатору getElementById



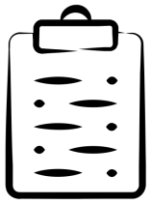
Также широко распространенный метод `getElementById`, селектор # в строке уже не нужен.

`getElementById('some')` и `querySelector('#some')` дадут одинаковый результат

```
<meta charset="UTF-8">
<body>
    <div id="div1">
        <span>Пример 1</span>
    </div>
</body>
```

```
> document.getElementById('div1')
  ▶ <div id="div1">…</div>
> document.querySelector('#div1')
  ▶ <div id="div1">…</div>
```

JS. Получение элементов. querySelectorAll



- Метод `querySelectorAll()` возвращает все элементы (Elements) документа, которые соответствует указанному селектору или группе селекторов в виде коллекции NodeList.

```
<meta charset="UTF-8">
<body>
    <div class="ex">Пример 1</div>
    <span>Пример 2</span>
    <div id="someItem">Пример 3</div>
    <span class="ex">Пример 4</span>
    <br/>
    <input type="text" value="Пример 5">
</body>
```

```
document.querySelectorAll('div')
document.querySelectorAll('span')
document.querySelectorAll('.ex')
```

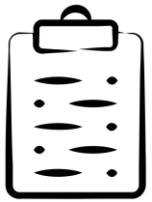
```
> document.querySelectorAll('div')
< ▶ NodeList(2) [div.ex, div#someItem]
```

```
> document.querySelectorAll('span')
< ▶ NodeList(2) [span, span.ex]
```

```
> document.querySelectorAll('.ex')
< ▶ NodeList(2) [div.ex, span.ex]
```

JS. Получение элементов. querySelectorAll

НТЦ ПОЛИТЕХНИКА



- Полученную коллекцию можно перебирать в цикле и обращаться по индексу, но эта коллекция не является стандартным массивом.

```
<div id="parent">
  <div>Пример 1</div>
  <div>Пример 2</div>
  <div>Пример 3</div>
</div>

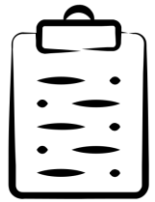
<script>
  document
    .querySelector('#parent')
    .querySelectorAll('div')
    .forEach(item => console.log(item.innerText));

  console.log(document.querySelectorAll('div')[2]);
</script>
```

```
Пример 1
Пример 2
Пример 3
<div>Пример 2</div>
>
```

JS. Получение элементов. querySelectorAll

НТЦ ПОЛИТЕХНИКА



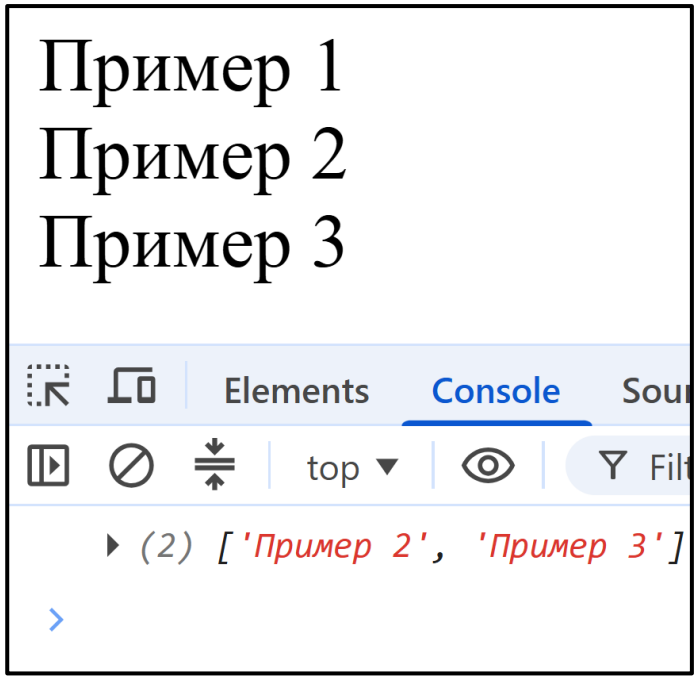
- Коллекцию NodeList можно преобразовать к массиву через оператор деструктуризации [...list]

```
<div id="parent">
  <div>Пример 1</div> <div>Пример 2</div> <div>Пример 3</div>
</div>

<script>
  const nodes = document
    .querySelector('#parent')
    .querySelectorAll('div');

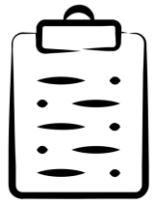
  const newList = [...nodes]
    .filter(item => +item.innerText.split(' ')[1] > 1)
    .map(item => item.innerText);

  console.log(newList);
</script>
```



JS. Поиск вверх по иерархии. closest

НТЦ ПОЛИТЕХНИКА



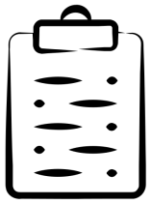
- Метод .closest() в JavaScript позволяет нам получить ближайший родительский элемент, который соответствует заданному CSS-селектору.

```
<body>
  <div class="ex">
    <div id="someItem">
      <span id="some">Пример 4</span>
    </div>
  </div>
</body>
```

```
> document.getElementById('some').closest('.ex')
<- ▶ <div class="ex">…</div>
> document.getElementById('some').closest('#someItem')
<- ▶ <div id="someItem">…</div>
```

JS. innerHTML

НТЦ ПОЛИТЕХНИКА

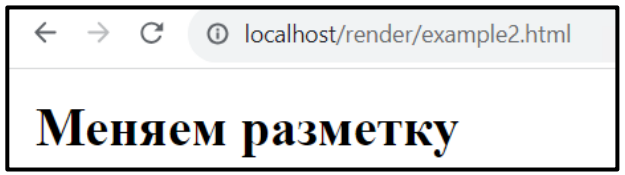


- Свойство innerHTML позволяет считать содержимое элемента в виде HTML-строки или установить новый HTML.
- Новое значение HTML необходимо передавать в виде строки и оно заменит текущее содержимое элемента.

```
<meta charset="UTF-8">
<body>
    <div id="div1">
        <span>Пример 1</span>
    </div>
</body>
```

```
> let elem = document.querySelector('#div1');
< undefined
> elem.innerHTML
< '\n        <span>Пример 1</span>\n    '
```

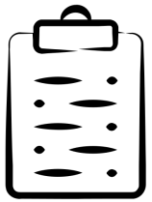
```
> elem.innerHTML = '<b>Меняем разметку</b>'
< '<b>Меняем разметку</b>'
```




```
<html> == $0
  <head> ... </head>
  <body>
    <div id="div1">
      <b>Меняем разметку</b>
    </div>
  </body>
</html>
```

JS. innerText и textContent

НТЦ ПОЛИТЕХНИКА



- Свойство innerText возвращает текст в том виде, в каком его видит пользователь на экране (с учетом стилей CSS)
- Свойство textContent возвращает «сырое» текстовое содержимое всех узлов, включая скрытые элементы и теги

```
<meta charset="UTF-8">
<body>
    <div id="div1">
        <span>Пример 1</span>
    </div>
</body>
```

```
> const elem = document.querySelector('#div1');
< undefined
> elem.innerText
< 'Пример 1'
> elem.textContent
< '\n    Пример 1\n  '
```

JS. Сравнение работы с контентом

НТЦ ПОЛИТЕХНИКА

Критерий	innerHTML	innerText	textContent
Что возвращает	HTML-разметку элемента	Видимый текст	Весь текст внутри элемента
Скрытый текст (display: none)	Учитывает (содержимое сохраняется)	Игнорирует	Учитывает
Теги <script> и <style>	Возвращает вместе с разметкой	Обычно игнорирует их содержимое	Возвращает их текстовое содержимое
Учет форматирования	Сохраняет HTML-теги	Учитывает визуальное форматирование, переносы строк	Не учитывает визуальное форматирование, переносы строк из CSS
Производительность	Может быть медленнее при обработке HTML	Медленнее, так как учитывает стили и layout	Обычно самый быстрый

JS. Сравнение работы с контентом

НТЦ ПОЛИТЕХНИКА

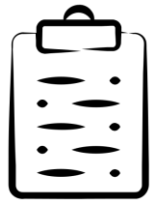
```
<div id="sample">
  One, <span style="display: none;">Two</span> Three!
  <style>p {background: green;}</style>
</div>
```



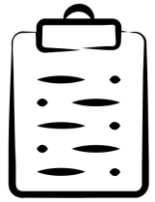

```
> const elem = document.querySelector('#sample');
< undefined
> elem.innerHTML
< '\n  One, <span style="display: none;">Two</span> Three!\n  <style>p {background: green;}</style>\n'
> elem.innerText
< 'One, Three!'
> elem.textContent
< '\n  One, Two Three!\n  p {background: green;}\n'
```

JS. Другие свойства и методы элементов

НТЦ ПОЛИТЕХНИКА



- Свойство classList для работы с css-классами (методы add, remove, toggle, contains)
- Свойство style для задания css-стиля сразу в атрибуте
- Свойство childNodes для дочерних тэгов
- Методы setAttribute и getAttribute для записи и чтения атрибутов тэга

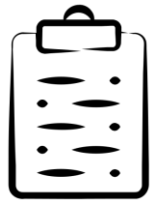


- Стиль, например, можно задать и через атрибуты:

```
div.setAttribute('style', 'color: red;');
```

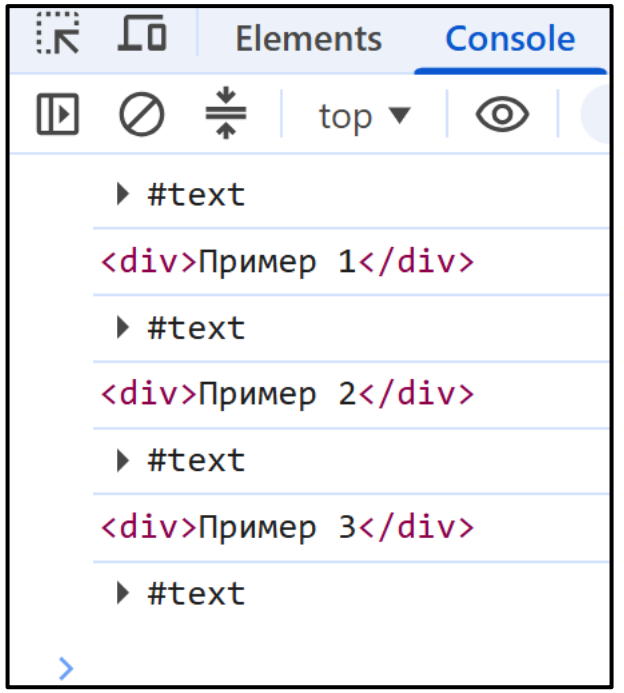
JS. Коллекция дочерних элементов childNodes

НТЦ ПОЛИТЕХНИКА



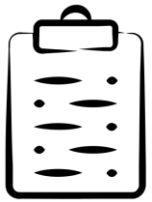
- Свойство childNodes возвращает коллекцию всех вложенных элементов без прохода вглубь по иерархии
- Часто цепляет элементы форматирования текста и выводит их как Text, что может потребовать дополнительной фильтрации

```
<div id="parent">
 <div>Пример 1</div>
 <div>Пример 2</div>
 <div>Пример 3</div>
</div>
<script>
  document
    .querySelector('#parent')
    .childNodes
    .forEach(item => console.log(item));
</script>
```



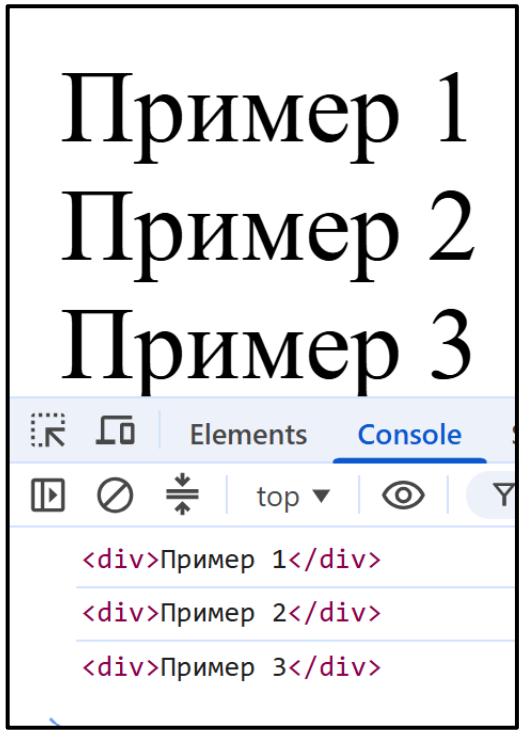
JS. Коллекция дочерних элементов children

НТЦ ПОЛИТЕХНИКА



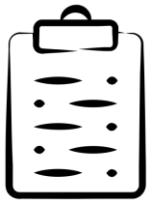
- Свойство children возвращает коллекцию HTMLCollection всех вложенных элементов без прохода по иерархии без текстовых узлов и комментариев
- Коллекция не является массивом, требуется приведение через [...] даже для forEach

```
<div id="parent">
<div>Пример 1</div>
<div>Пример 2</div>
<div>Пример 3</div>
</div>
<script>
  const childs = document
    .querySelector('#parent')
    .children;
  [... childs].forEach(item => console.log(item));
</script>
```



JS. Сравнение childNodes и querySelectorAll

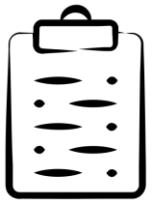
НТЦ ПОЛИТЕХНИКА



- childNodes возвращает только прямых потомков (первый уровень вложенности) конкретного узла, но включает в себя все типы узлов: элементы, текст (включая пробелы и переносы строк) и комментарии
- querySelectorAll ищет элементы на всю глубину вложенности (внутри всей ветки дерева), но возвращает только узлы-элементы (теги), которые соответствуют заданному CSS-селектору
- querySelectorAll возвращает статический снимок, размерность который не меняется, а childNodes возвращает коллекцию по ссылке, которая может менять свой размер и наполнение

JS. Создание элемента

НТЦ ПОЛИТЕХНИКА

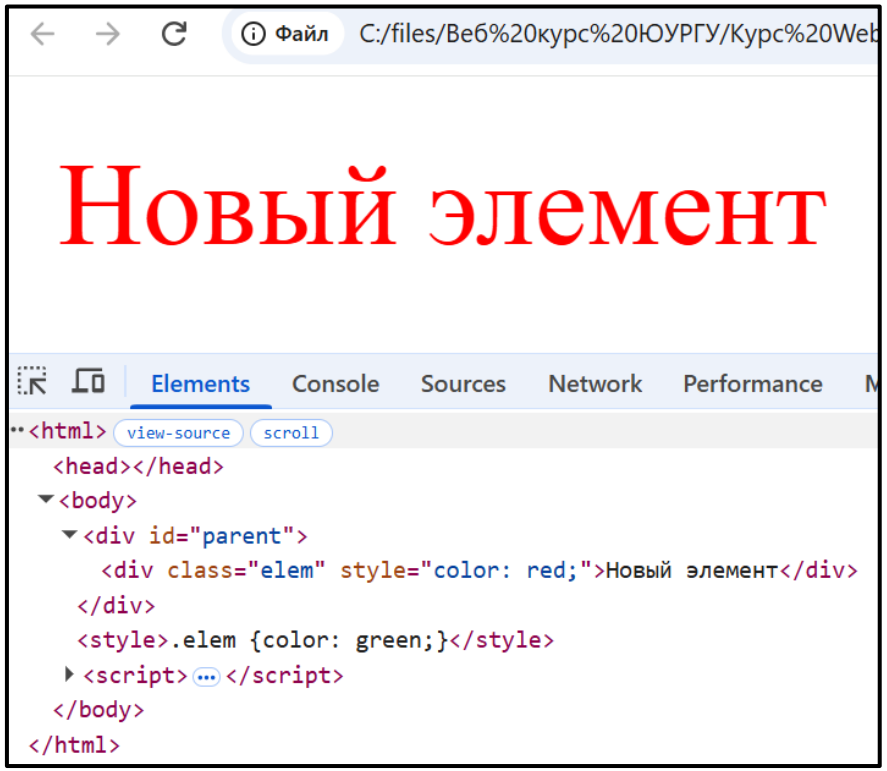


- Создание элемента происходит по вызову метода document.createElement
- Чтобы добавить в родительский элемент, нужно вызвать у родительского элемента append или appendChild.

```
<div id="parent"></div>
<style>.elem {color: green;}</style>

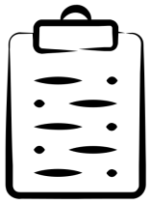
<script>
  const div = document.createElement('div');
  div.style = 'color: red;';
  div.textContent = 'Новый элемент';
  div.classList.add('elem');

  document
    .getElementById('parent')
    .append(div);
</script>
```



JS. Сравнение подходов создания элементов

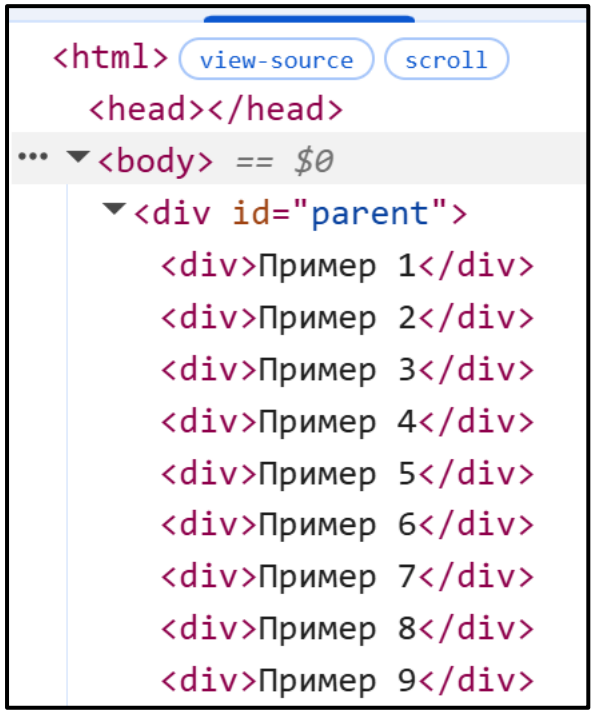
НТЦ ПОЛИТЕХНИКА



- В текущем варианте мы создаем элементы в цикле по очереди, что может быть медленно на очень больших объемах

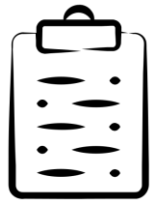
```
<div id="parent"></div>
<script>
  const parent = document.getElementById('parent');

  for (let i = 0; i < 100; ++i) {
    const elem = document.createElement('div');
    elem.innerText = `Пример ${i+1}`;
    parent.append(elem);
  }
</script>
```



JS. Сравнение подходов создания элементов

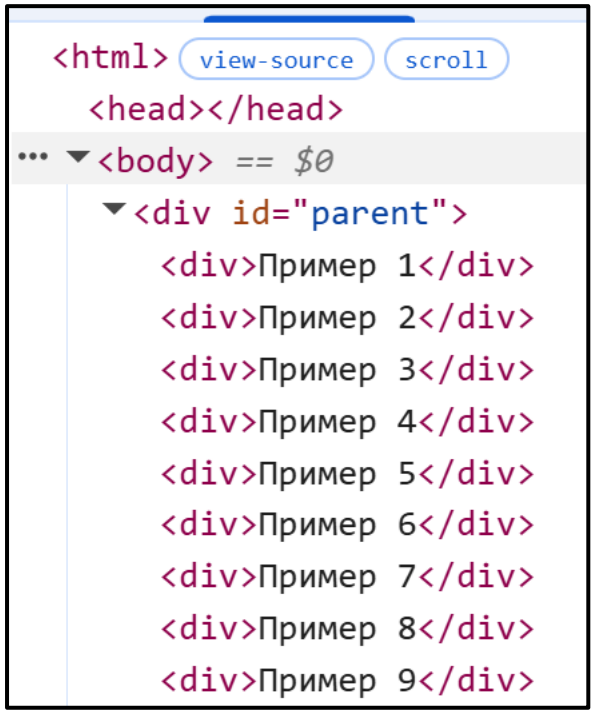
НТЦ ПОЛИТЕХНИКА



- В этом варианте мы формируем элементы через добавление к строке, что тоже медленно, т.к. при каждом изменении строки JS сохраняет ее ОЗУ заново

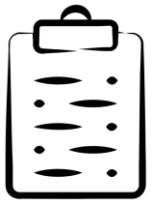
```
<div id="parent"></div>
<script>
  const parent = document.getElementById('parent');

  for (let i = 0; i < 100; ++i) {
    parent.innerHTML += `<div>Пример ${i+1}</div>`;
  }
</script>
```



JS. Сравнение подходов создания элементов

НТЦ ПОЛИТЕХНИКА



- В этом варианте мы формируем массив строк, в конце объединяем массив в 1 строку через join и рендерим все дерево одной вставкой в innerHTML

```
<div id="parent"></div>
<script>
  const parent = document.getElementById('parent');
  const childs = [];

  for (let i = 0; i < 100; ++i) {
    childs.push(`<div>Пример ${i+1}</div>`);
  }

  parent.innerHTML = childs.join('');
</script>
```

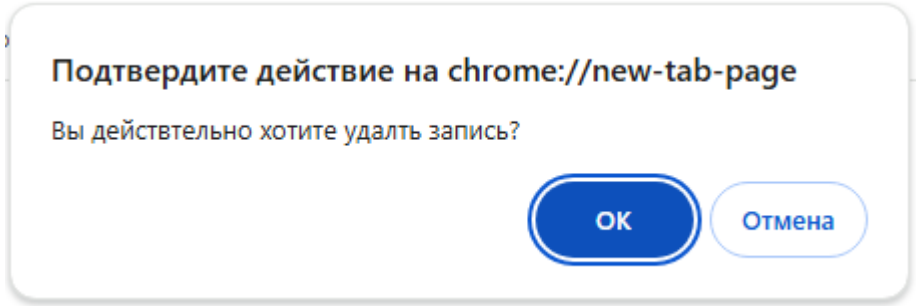
```
<html> view-source scroll
  <head></head>
... <body> == $0
    <div id="parent">
      <div>Пример 1</div>
      <div>Пример 2</div>
      <div>Пример 3</div>
      <div>Пример 4</div>
      <div>Пример 5</div>
      <div>Пример 6</div>
      <div>Пример 7</div>
      <div>Пример 8</div>
      <div>Пример 9</div>
```

JS. Подтверждение действия от пользователя

НТЦ ПОЛИТЕХНИКА

Для подтверждения действия можно использовать функцию `confirm('сообщение')`.

Она синхронно ожидает действие пользователя. Если пользователь нажал «Ок», вернет true, если «Отмена», вернет false.



«Ок»

```
> confirm('Вы действтельно хотите удалть запись?')
< true
```

«Отмена»

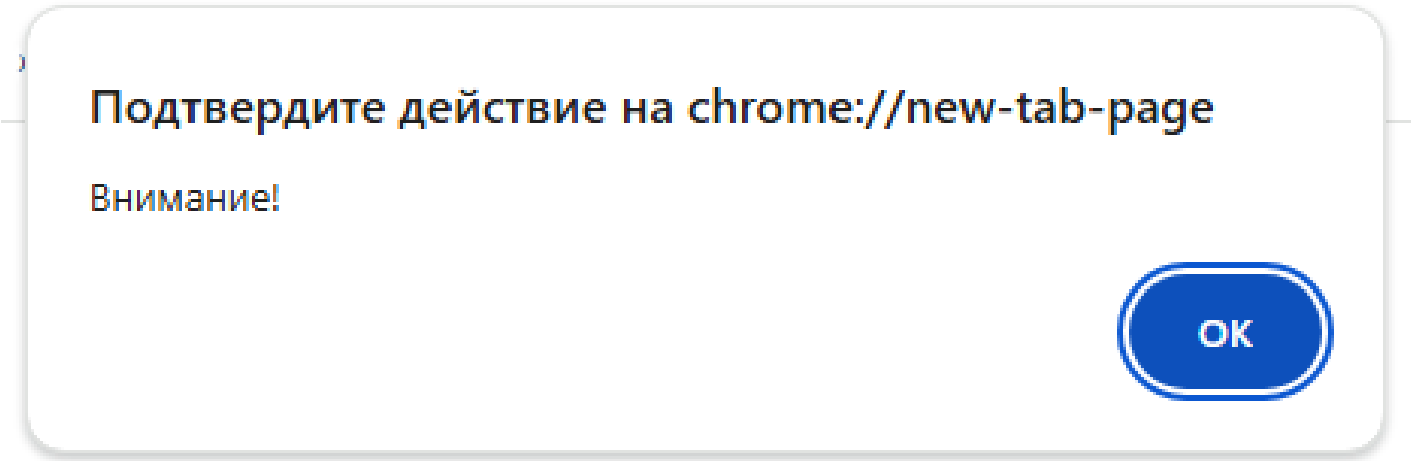
```
> confirm('Вы действтельно хотите удалть запись?')
< false
```

JS. Вывод сообщений пользователю

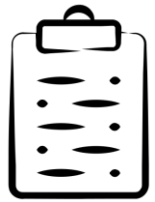
НТЦ ПОЛИТЕХНИКА

Для вывода сообщения пользователю можно использовать функцию `alert('сообщение')`:

```
> alert('Внимание!')
< undefined
```



JS. События

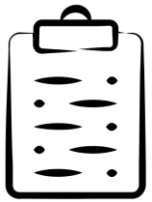


Событие – это сигнал от браузера о том, что что-то произошло. Все DOM-узлы подают такие сигналы, например:

- click – происходит, когда кликнули на элемент левой кнопкой мыши (на устройствах с сенсорными экранами оно происходит при касании).
- contextmenu – происходит, когда кликнули на элемент правой кнопкой мыши.
- mouseover / mouseout – когда мышь наводится на / покидает элемент.
- mousedown / mouseup – когда нажали / отжали кнопку мыши на элементе.
- mousemove – при движении мыши.

JS. События. onclick

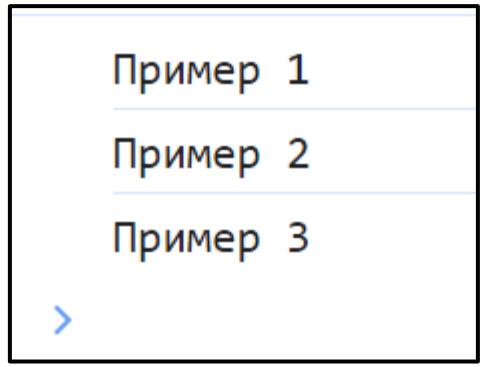
НТЦ ПОЛИТЕХНИКА



Назначить обработчик события можно либо в соответствующем атрибуте, либо непосредственно в JS

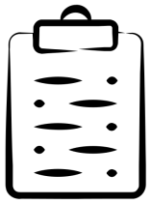
```
<meta charset="UTF-8">
<body>
    <div id="div1">Пример 1</div>
    <div onclick="console.log('Пример 2')">Пример 2</div>
    <div onclick="myFunc()">Пример 3</div>

    <script>
        const item = document.querySelector('#div1')
        item.onclick = function() {
            console.log('Пример 1')
        }
        function myFunc() {
            console.log('Пример 3')
        }
    </script>
</body>
```



JS. Иерархия событий

НТЦ ПОЛИТЕХНИКА

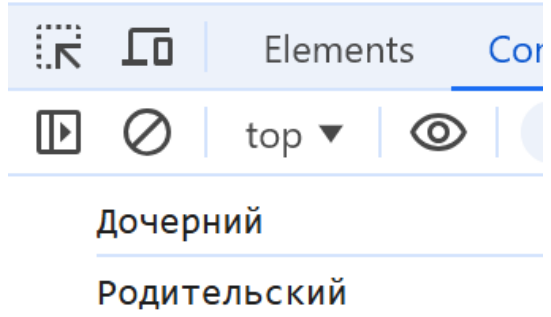


Если на все элементы наложены события, например, onclick, по умолчанию они будут вызваны, начиная с самого дочернего.

- Браузер проверяет, имеет ли элемент, который был фактически нажат, обработчик события onclick, зарегистрированный на нем в фазе всплытия, и запускает его, если это так.
- Затем он переходит к следующему непосредственному родительскому элементу и выполняет то же самое, затем следующее и так далее, пока не достигнет элемента <html>.

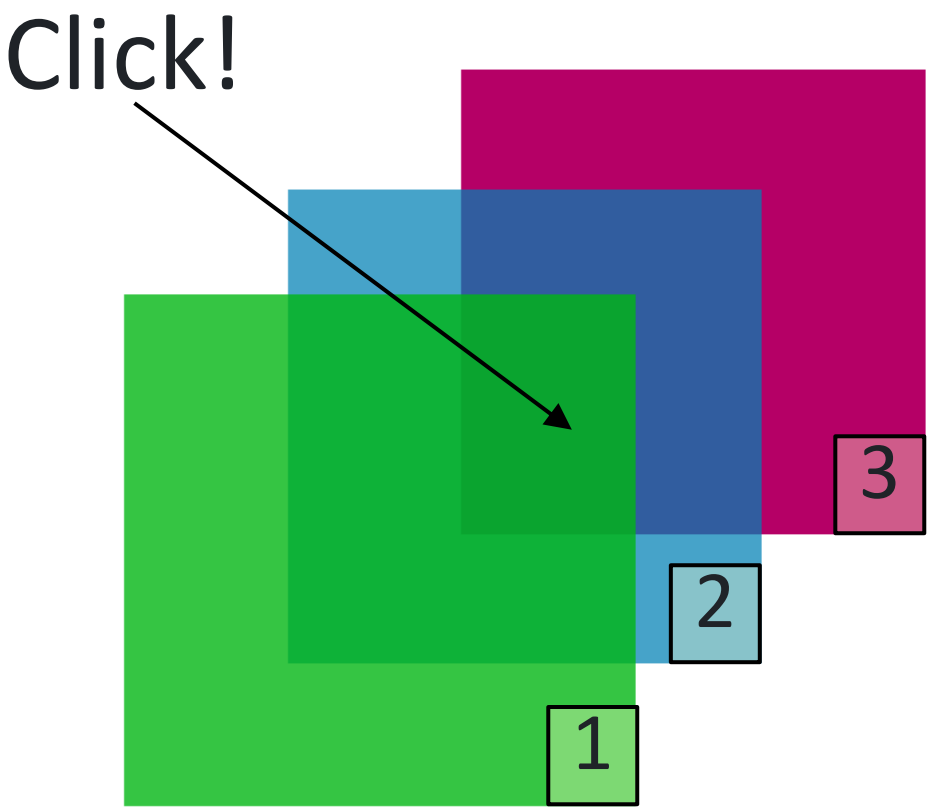
```
<meta charset="UTF-8">
<body>
    <div onclick="console.log('Родительский')">
        <div onclick="console.log('Дочерний')">
            Текст
        </div>
    </div>
</body>
```

Текст



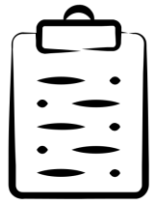
JS. Иерархия событий

По умолчанию используется «всплытие», от дочернего к родительскому



1. Событие 1
2. Событие 2
3. Событие 3

JS. Объект событий event

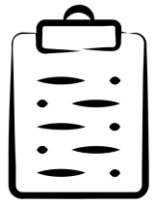


В каждое событие передается объект event, который содержит информацию о текущем событии. Структура каждого event для каждого события отличается, необходимо обращаться к документации и стандарту.

```
<meta charset="UTF-8">
<body>
    <div onclick="console.log(event)">
        Текст
    </div>
</body>
```

```
▼ PointerEvent {isTrusted: true, pointerId: 1, wic
    isTrusted: true
    altKey: false
    altitudeAngle: 1.5707963267948966
    azimuthAngle: 0
    bubbles: true
    button: 0
    buttons: 0
    cancelBubble: false
    cancelable: true
    clientX: 34
    clientY: 19
    composed: true
    ctrlKey: false
    currentTarget: null
    defaultPrevented: false
    detail: 1
    eventPhase: 0
    fromElement: null
```

JS. Объект событий event



Помимо свойств у event есть и методы. Самые часто используемые:

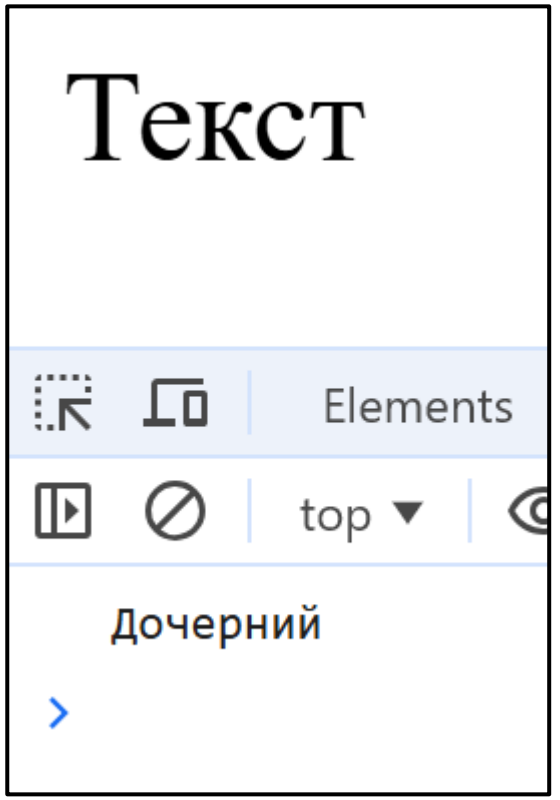
- `stopPropagation()` – остановить всплытие события. Если у родительского элемента есть события того же типа, то он не будут вызваны
- `preventDefault()` – остановить действие по умолчанию (переходы по ссылке, меню на ПКМ, отмена печати и т.д.)

JS. Объект событий event. stopPropagation()

НТЦ ПОЛИТЕХНИКА

```
<meta charset="UTF-8">
<body>
    <div id="div1">
        <div id="div2">
            Текст
        </div>
    </div>
    <script>
        document.querySelector('#div1').onclick = () => {
            console.log('Родительский');
        }

        document.querySelector('#div2').onclick = (event) => {
            // Событие на div1 не уйдет
            event.stopPropagation();
            console.log('Дочерний');
        }
    </script>
</body>
```



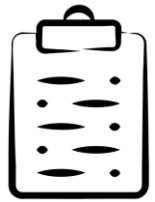
JS. Объект событий event. stopPropagation()

НТЦ ПОЛИТЕХНИКА

```
<meta charset="UTF-8">
<body>
    <a href="https://developer.mozilla.org/">
          Текст
    </a>
    <script>
        document.querySelector('a').onclick = (event) => {
            // Переход по ссылке не произйодет
            event.preventDefault();
        }
    </script>
</body>
```



JS. data-атрибут элементов html



В HTML тегах можно задавать пользовательские атрибуты, которые начинаются с data-*.

Это позволяет хранить внутри тегов некоторые данные с привязкой к этому тегу.

```
<article
  id="electriccars"
  data-columns="3"
  data-index-number="12314"
  data-parent="cars">
  ...
</article>
```

JS. data-атрибут элементов html

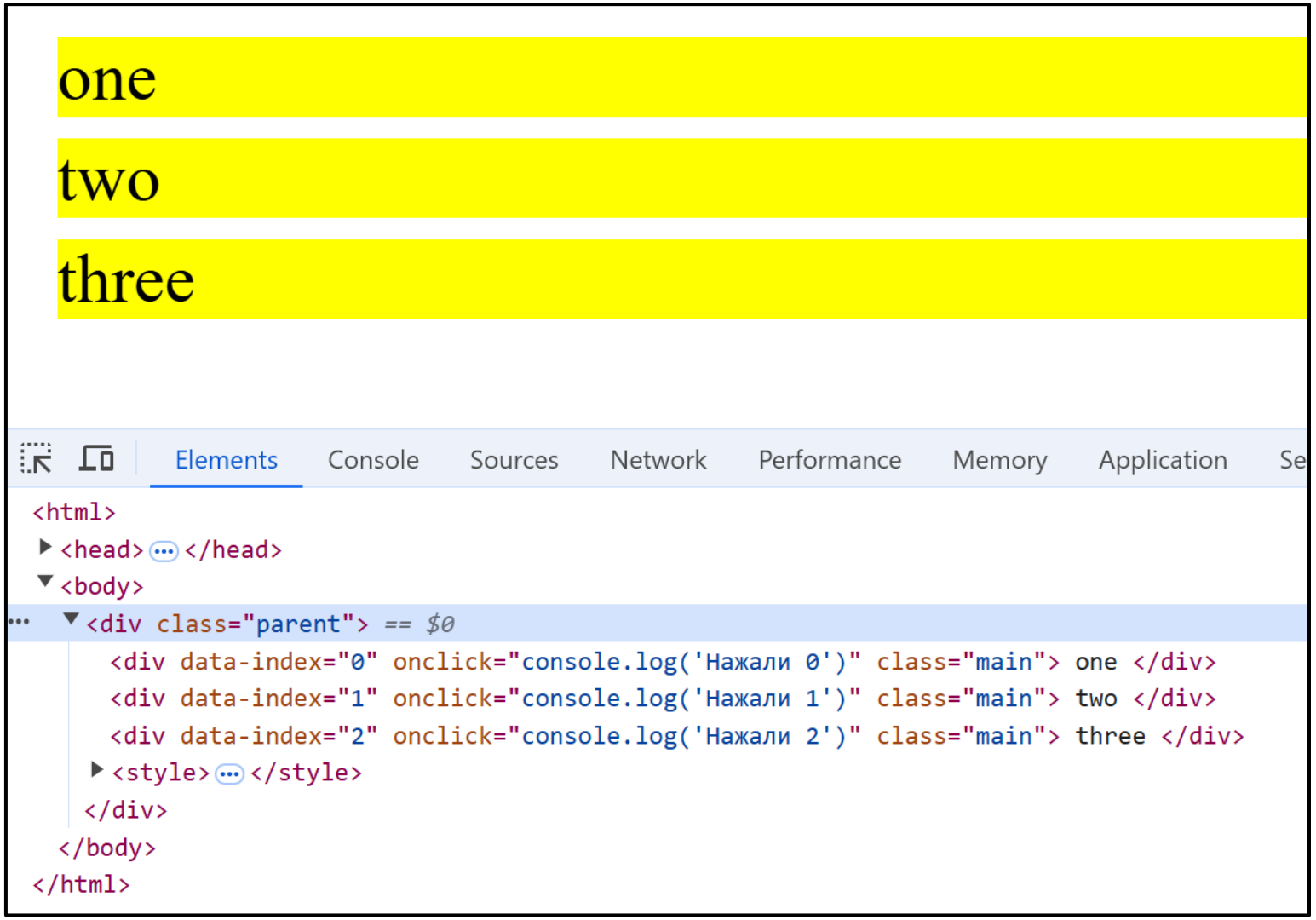
НТЦ ПОЛИТЕХНИКА

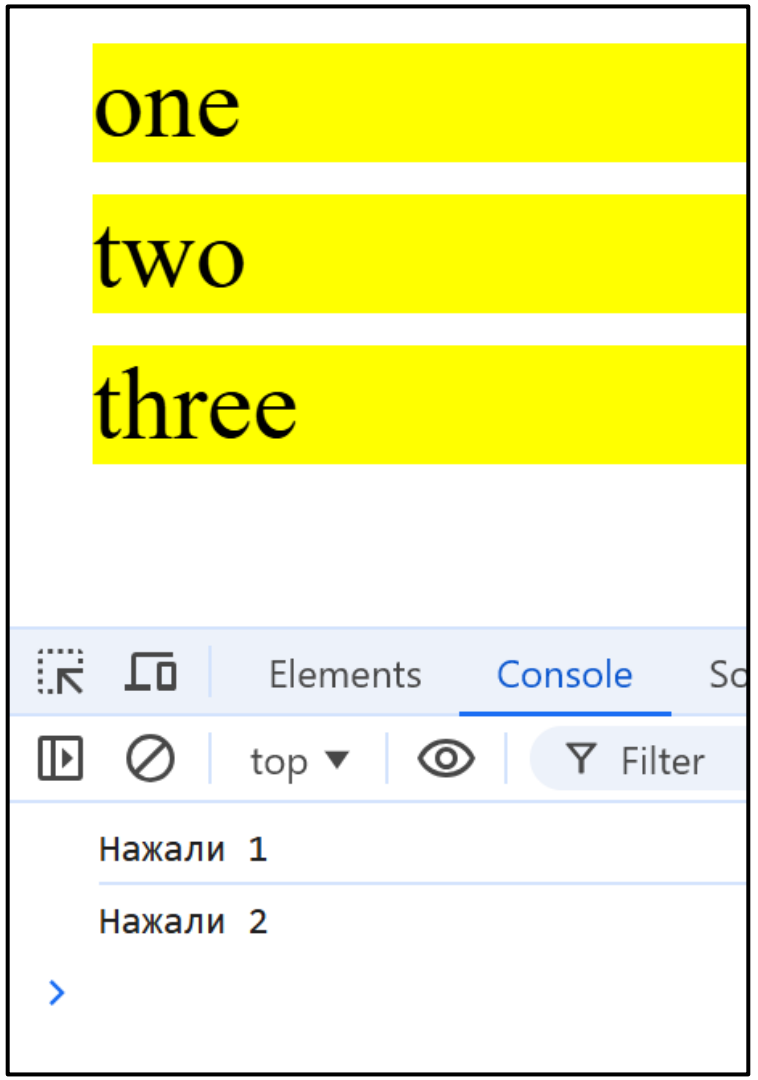
```
<meta charset="UTF-8">
<body>
    <div class="parent"/>
    <script>
        const data = ['one', 'two', 'three'];
        const domList = data.map((value, index) =>
            `<div data-index="${index}"
                  onclick="console.log('Нажали ${index}')"
                  class="main">
                ${value}
            </div>`
        );

        const parent = document.querySelector('.parent');
        parent.innerHTML = domList.join('');
    </script>
    <style>
        .main {
            margin: 5px;
            background: yellow;
        }
    </style>
</body>
```

JS. data-атрибут элементов html

НТЦ ПОЛИТЕХНИКА





JS. data-атрибут элементов html

НТЦ ПОЛИТЕХНИКА

```
<script>
    const data = [
        {
            id: 1111, value: 'one'
        },
        {
            id: 2222, value: 'two'
        },
        {
            id: 3333, value: 'two'
        },
    ];
    const domList = data.map(item =>
        `<div data-id="${item.id}"
              class="main">
            ${item.value}
        </div>`
    );

    const parent = document.querySelector('.parent');
    parent.innerHTML = domList.join('');
</script>
```



Южно-Уральский государственный университет

Национальный исследовательский университет

НТЦ ПОЛИТЕХНИКА

технологии • решения • интеграция

Спасибо за внимание!



2026 ООО НТЦ «Политехника»

Челябинск, ул. Карла Маркса, д. 38